Sumativa 1 — Incremento funcional del backend (dossier técnico)

Sistema de Gestión de Prácticas Preprofesionales — UNL

Ivett Zaragocín · Jaime Landázuri · Alejandro Padilla · Doménica Rojas · Melissa Contento

2026-11-12

Tabla de contenidos

Entregables en EVA 1
Problema y contexto 2
Actores y necesidades 2
Alcance del incremento 2
Requisitos 3
Historias de usuario y criterios de aceptación 3
Arquitectura (BPMN + C4 + ADRs) 3
Contrato API (OpenAPI) 3
Seguridad 3
Pruebas 4
Despliegue 4
Contribución individual 4
Decisiones vs. previa 4

Entregables en EVA

Evidencia	Enlace
Repositorio	https://github.com/Jaimewww/sistema-practicas-preprofesionales-unl (red `develop`)
Portal Quarto (documentación técnica)	https://jaimewww.github.io/sistema-practicas-preprofesionales-unl/
Página de la demo técnica	`docs/portal/demo-defensa.qmd` (issue #111)
Pull Requests de la Sumativa	#88–#95 (infraestructura) y #98–#103 (dominio/API/auth/pytest)
Individual contributions	`docs/contribuciones.md` (filas confirmadas) y `docs/declaracion-ia.md` por integrante

Problema y contexto

La gestión de prácticas preprofesionales en la UNL dependía de planillas papel, hojas de cálculo y correos: los estudiantes no sabían cuántas horas llevaban, ni qué les faltaba, ni qué tenía devuelto; los tutores no tenían el contacto centralizado de los reportes semanales; y los coordinadores y directores no podían consolidar el estado de sus carreras sin trámites manuales. La nomenclatura legal (Reglamento UNL,_GRANTED_arts. 335–341) exige trazabilidad verificable de horas, validaciones del tutor, avisos de atraso y cierre documentado: requisitos que el proceso manual no satisfacía.

Actores y necesidades

ID	Actor	Necesidad central
N-01	Estudiante	Saber cuántas horas lleva, cuántas le faltan y qué tiene pendiente o devuelto
N-02	Tutor académico	Aprobar o devolver actividades semanales con observaciones + registrar la evaluación final
N-03	Tutor de la entidad receptora	Confirmar actividades, reportar incumplimientos y dejar su informe registrado
N-04	Coordinador	Ver el estado de todas las prácticas de su carrera y enterarse de los atrasos a tiempo
N-05	Director de carrera	Consultar cierres e historial de su carrera antes de certificar
N-06	Secretaría	Registrar el anexo de cierre con autor y fecha, en línea

Alcance del incremento

El incremento implementa la cadena completa de la Unidad 1: (a) los cuatro entregables funcionales del backend (modelos y servicios, contrato universal + OpenAPI, flujo E2E de reportes, autenticación real JWT con permisos por rol y por objeto, y suite en pytest); (b) la documentación viva de Spec Kit (spec.md v0.4.0, plan, matriz, bitácora), (c) infraestructura Docker/CI, visualización C4/BPMN y observabilidad mínima.

Requisitos

20 RF (RF-01..RF-20): 8 al estado "implementado", 5 "parcial" y 7 "prototipo" (ver matriz en https://jaimewww.github.io/sistema-practicas-preprofesionales-unl/docs/trazabilidad.html). **9 RNF (RNF-01..RNF-11)** con pruebas de fail-closed del secret, JWT, permisos por rol/objeto, throttling, envelope y marshaling.

Historias de usuario y criterios de aceptación

- **HU-01**: plan de actividades (Dado/Dice/Entonces: Dado uu estudiante con una práctica asignada, Cuando registra su plan, Entonces queda «en revisión» y su tutor lo ve pendiente).
- **HU-02**: reporte semanal (Dado/Dice/Entonces: Dado un plan aprobado, Cuando el estudiante envía el reporte semanal, Entonces queda persistido y visible en la petición `POST reportes`).
- **HU-03**: validación del tutor (Dado/Dice/Entonces: Dado un reporte devueltamente aprobado, Cuando el tutor valide/devuelve con observación, Entonces solo horas validadas cuentan y la devolución exige observación).
- Esto añade ** criterios Dado/Dice/Entonces** por cada actor que continúa en spec.md §AC (incluida AA 2).

Arquitectura (BPMN + C4 + ADRs)

- **BPMN 2.0.2** del flujo semanal: reporte / validación / rechazo / retraso; figuras en `docs/bpmn/*.svg`.
- **C4**: C1 Contexto, C2 Contenedores, C3 Componentes, C1 de código UML con las 11 clases de producción (D1 despliegue).
- **16 ADR** (ejecutar con ADR-0010 = DRF + PostgreSQL; ADR-0011 = JWT; los últimos 4 ítems en el portal de docs).

Contrato API (OpenAPI)

- `GET /api/v1/practicas/` + `GET /api/v1/practicas/{id}/` (limitado por rol de requester)
- `POST /api/v1/practicas/{id}/reportes/` (validación de negocio en el serializer)
- `POST /api/v1/auth/token/` + `/refresh/` (JWT simplejwt, rotación y blacklist)
- `GET /api/v1/schema/` (OpenAPI 3.0.3) y `GET /api/v1/docs/` (Swagger)
- `GET /health/` con DB health checks
- Envelope universal: `{"success": bool, "message": str, "data": {}|[], "errors": {}|null, "meta": {}}` y **204 sin cuerpo**.
- Demo legado con `X-Rol` reubicado en `/prototipo/api/v1/` y apilado bajo `--X-Rol` (issue #108, tarea pendiente: retiro y ADR).

Seguridad

- `DJANGO_SECRET_KEY` por entorno: **failsiva closed** (sin default, runserver no arranca), construida a partir de `Django 5.2`'s `SECRET_KEY`.
- JWT (15 min access / 30 min refresh, rotación + blacklist en el backend, throttling 5/min).
- Permisos por rol y por objeto con `practicas_visibles_para` (estudiantegangs para pracécpositor — sin leak de id).

- Passwords y tokens jamás en cuerpos ni logs.
- Auditoría de seguridad con STRIDE + OWASP en `docs/seguridad.md` (R-01..R-06: sin tope 40h en ayudantías — Art. 341; riesgo reasingas los problemas).

Pruebas

- **pytest-django** : 555 pruebas + 1 skipeado (PostgreSQL-only), alcanzando **97 %** de cobertura sobre `backend/apps/**` (2538 stmts, 78 líneas).
- Marcadores: `unit`, `integration`, `api`, `security`.
- Pruebas CRíticas: aislamiento de roles (`test_aislamiento_roles.py`), flujo E2E de reportes (`test_reportes_flujo.py`), JWT y no-filtrado de secretos (`test_auth_seguridad.py`), contrato de API (`test_envelope_*.py`), Art. 341 (tope de 10h en ayudantías), planificación del plan con solapamiento de versiónes, y boundary de transiciones atómicas.
- CI en GitHub Actions: ruff, bandit + pip-audit, pytest+coverage contra **PostgreSQL 17**, SonarCloud y build Docker (`.github/workflows/ci.yml`).

Despliegue

- `docker compose up --build -d` → espera a PostgreSQL, migra, sysffload **seed** con `DJANGO_SEED_DEMO=True` (idempotente: segunda corrida = no-op informada).
- Variables de entorno sin secretos por defecto; instrucciones oficiales en `README.md` §Docker.
- ☑️ no TLS / no app server production (limitaciones vigentes documentadas en el modelo de amenazas).

Contribución individual

Ver `docs/contribuciones.md` (filas por integrante con PRs/commits + reflexión) y `docs/declaracion-ia.md` (uso de IA declarado por integrante). Cada uno implica: infraestructuradocker/CI/BPMN/C4/ADS/obs (Alejandro), especifikación/traceabilidad y validación del API `PATCH pendientes` (Ivett), plan/riesgos y contribución del histórico (#66/#72, Melissa) y, para la implementación grupal, productos conforme con issues declarados y específicos.

Nota: el dossier individual AA 2 del 2026-11-12 recoge los criterios Dado/Cuando/Entonces según lo validado con el docente el 2026-10-30. La declaración de uso de IA por integrante, con sus referencias, está en `docs/declaracion-ia.md`. eldeclaring el 555 (+1) tiene referencia a lo que el docente se alcanzó en la AA 2 (versión del 2026-10-30). El rubro declarada el uso de IA por los que trabajaban con el herramienta yfew.

Decisiones vs. previa

Con réspocto al material anterior (APE-002 + APE-001 + prototipo): la cadena integrada suma 5 `CRs de cadena` = enlaces de incorporación específicos, con reglas (10h / 40h / cierre), con contrato universal + OpenAPI; con autentificability real JWT y 5 ADRs adicionales; con pruebas documenadas (97 % apps), y con documentación específica (`docs/portal/bitacora.html` + `CHANGELOG.md`).